

SHARP PC-1251/PC-1245 エミュレータ

使い方の手引き



おりぐち

2026 年 10 月 9 日 (エミュレータ 0.3.0 版)

このエミュレータは PC-1251 と PC-1245 の ROM をそのまま動かします。ROM は付属していません (2.4 節)。

目次

1	はじめに	2
2	入れ方と起動	2
2.1	端末を開く	2
2.2	uv を入れる	3
2.3	エミュレータのフォルダを用意する	3
2.4	ROM を用意する	4
2.5	起動する	5
2.6	終わる	5
2.7	うまく動かないとき	6
3	画面と操作	7
3.1	各部の名前	7
3.2	マウスで操作する	7
3.3	パソコンのキーボードで操作する	8
3.4	エミュレータの操作	8
3.5	モードスイッチ	10
3.6	SHIFT と DEF	10
4	使ってみる	10
4.1	電卓として使う	10
4.2	BASIC のプログラムを打ち込んで動かす	10
4.3	打ち込みを楽にする、行を直す・消す	11
4.4	PRINT で止めずに見せる (WAIT)	12
4.5	DEF キーで実行する	13
4.6	RSV モードで文字列を登録する	14
4.7	エラーが出たら	14
4.8	電源とメモリ	14
5	プログラムのファイルを使う	15
5.1	置き場所に置いて、一覧から読み込む	15
5.2	ファイルの形式	17
5.3	端末から読み込む	19
5.4	マシン語を動かす	19
5.5	CALL で表示をつけたまま計算する	19
5.6	マシン語で液晶に描く	20
5.7	PC-インタープリタともぐらたたき	22
5.8	ほかの読み込み方	22
5.9	プログラムをファイルに書き出す	23
5.10	カセットの音 (wav) で実機とやり取りする	23
6	PC-1245 として使う	24
6.1	液晶	25
6.2	モードスイッチ	25
6.3	キー	25
6.4	メモリと見本のプログラム	26
付録 A	起動のオプション	26

付録 B	SHIFT で入る記号	27
付録 C	メモリマップと入出力	27
付録 D	実機と違うところ	28
付録 E	ファイルの一覧	28

1 はじめに

このエミュレータは、シャープのポケットコンピュータ PC-1251(1982 年発売、29,800 円) と PC-1245(1983 年発売、17,800 円) を、パソコンの上で動かすものです。CPU の SC61860 を命令単位で動かし、実機の内部 ROM と BASIC ROM をそのまま実行します。キーとモードスイッチはマウスでも操作できます。

BASIC のプログラムを打ち込んで動かせるほか、電卓としても使えます。BEEP の音も鳴ります。BASIC のテキストや 16 進ダンプのファイルを決まったフォルダに置いておけば、一覧から選んで読み込めますし、マシン語を RAM に書いて CALL で呼ぶこともできます。実機の「MEMORY SAFE GUARD」と同じく、終了してもプログラムは消えません。

どちらの機種にするかは起動のときに選び、何も指定しなければ PC-1251 になります。この手引きは PC-1251 を例に書き、PC-1245 で違うところは 6 節にまとめました。

動作環境

- macOS、Windows(10 以降)、Linux。
- Python 3.10 以降。Python を自分で入れる必要はありません。次の節で入れる uv が、必要な版の Python とライブラリをまとめて用意します。
- 初回の起動でライブラリを入れるときだけ、インターネットにつながっている必要があります。

キーとコマンドの書き方

パソコンのキーは `Esc`、本体のキーは `SHIFT` のように書き分けます。`command`/`Ctrl`+`O` は、macOS では `command` を、Windows と Linux では `Ctrl` を押しながら `O` を押すという意味です。macOS と Windows で違うところは「macOS」「Windows」と見出しをつけて両方を書きました。端末 (macOS の「ターミナル」、Windows の「PowerShell」) で打つコマンドは、次のような枠に入れています。行の頭の `$` や `>` は打たなくてかまいません。

```
$ uv run pc1251
```

2 入れ方と起動

2.1 端末を開く

■ macOS 「アプリケーション」→「ユーティリティ」→「ターミナル」を開きます。Spotlight(`command`+`Space`) で「ターミナル」と打つても開けます。

■Windows スタートメニューで「PowerShell」と打ち、「Windows PowerShell」を開きます。「ターミナル」(Windows Terminal)が入っていれば、「ターミナル」を使ってもかまいません。

2.2 uv を入れる

uv は Python とライブラリをまとめて扱う道具です。一度入れれば、次からは入れ直す必要はありません。

■macOS

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

■Windows

```
> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

入れ終わったら、端末をいったん閉じて開き直します(新しく入ったコマンドの場所を端末に読み込ませるため)。次のコマンドでバージョン番号が出れば成功です。

```
$ uv --version
```

Homebrew や winget を使っている場合

macOS で `brew install uv`、Windows で `winget install --id=astral-sh.uv -e` としても同じ uv が入ります。

2.3 エミュレータのフォルダを用意する

GitHub のページから `git clone` で取ってくるか、「Code」→「Download ZIP」でダウンロードした ZIP ファイルを展開して、好きな場所に置いてください。`git clone` で取ってきたときのフォルダの名前は `pc1251-emulator` で、以下でも `pc1251-emulator` と書きます。端末で `pc1251-emulator` に移ってからコマンドを打ちます。

■macOS (ホームの src の下に置いた場合)

```
$ cd ~/src/pc1251-emulator
```

■Windows (「ドキュメント」の下に置いた場合)

```
> cd $HOME\Documents\pc1251-emulator
```

フォルダの場所を打つのが面倒なら、`cd` と空白を打ってから、Finder やエクスプローラーのフォルダをターミナルや PowerShell のウィンドウにドラッグ&ドロップすると、場所が入ります。

2.4 ROM を用意する

ROM はシャープの著作物なので、エミュレータには含めていません。実機から読み出したものを使ってください。

機種ごとに2つのファイルが要ります。エミュレータのフォルダの中に `rom` フォルダを作り、表のファイル名で置きます (置き方の例は表のあと)。使う機種のぶんだけあれば動き、両方を並べて置いてもかまいません。

ファイル名	中身	大きさ
<code>cpu-1251.rom</code>	PC-1251 の CPU(SC61860) の内部 ROM、0000～1FFF 番地	8192 バイト
<code>bas-1251.rom</code>	PC-1251 の BASIC ROM、4000～7FFF 番地	16384 バイト
<code>cpu-1245.rom</code>	PC-1245 の CPU の内部 ROM、0000～1FFF 番地	8192 バイト
<code>bas-1245.rom</code>	PC-1245 の BASIC ROM、4000～7FFF 番地	16384 バイト

たとえば macOS でホームの `src` の下に置いた場合は、次のようになります。

```
~/src/pc1251-emulator/
  pc1251emu/
    programs/
      rom/
        cpu-1251.rom
        bas-1251.rom
        cpu-1245.rom      (PC-1245 も使うとき)
        bas-1245.rom
```

`rom` フォルダは、エミュレータのフォルダに移ってから次のコマンドで作れます (macOS でも Windows でも同じです)。Finder やエクスプローラーで ROM のファイルを `rom` フォルダへ入れます。

```
$ mkdir rom
```

Windows で「ドキュメント」の下に置いた場合は、`C:\Users\名前\Documents\pc1251-emulator\rom\cpu-1251.rom` のようになります。

実機からの読み出し方は、大阪工業大学のサイトにある yagshi 氏のページ「Extracting ROM Data (ROM 吸い出し)」(https://www.oit.ac.jp/rd/labs/kobayashi-lab/~yagshi/old_web/misc/pocketcom/extrom.html) に、PC-1251 を例にした手順があります。BASIC ROM は `CSAVE M` で区切りながらカセットに保存し、内部 ROM は短いマシン語で外に写してから保存します。録音した音をパソコンでバイナリに戻し、つなげて1つのファイルにします。PC-1245 は、ht-

deko.com のページ「SHARP PC-1245/50/51/55 の使い方」(<https://ht-deko.com/pc1245/>)に、BASIC のリストつぎの手順があります。内部 ROM は短いマシン語で 1KB ずつ RAM に写してから、BASIC ROM は 4KB ずつ、どちらも CSAVE M で保存します。

ROM には版の違いがあるので、エミュレータが確かめるのはファイルの大きさだけです。ファイルが見つからないときや大きさが足りないときは、起動したときにメッセージを出して終わります。

初めて uv run を使うとき

最初の uv run では、uv が Python と、使うライブラリ (pygame、Pillow、Typer) を取ってきて入れるので、しばらく時間がかかります。2 回目からはすぐに始まります。ライブラリはエミュレータのフォルダの中の .venv に入ります (Python の本体と、取ってきたファイルの控えは、uv が自分のフォルダにしています)。

ROM を rom 以外のフォルダに置くときは、環境変数 PC1251_ROM にフォルダの場所を入れておきます。ほかに、ホームの .pc1251 フォルダも探します。

2.5 起動する

```
$ uv run pc1251
```

PC-1251 の画面がウィンドウに出て、液晶に>(プロンプト) が出れば動いています。最初の 8 秒ほどは、画面の中ほどにキーの説明が出ます。何かキーを押すかクリックすると消えます (command / Ctrl + / でいつでも出し入れできます)。

PC-1245 として起動するときは、--model 1245 を付けます (6 節)。

```
$ uv run pc1251 --model 1245
```

画面が大きすぎたり小さすぎたりするときは、拡大率を指定します。画面に収まらないときは自動で縮めます。

```
$ uv run pc1251 --scale 0.8
```

2.6 終わる

ウィンドウを閉じれば終わります。終わるときには、実機のスイッチを OFF にしたのと同じ手順で電源を切ってから、RAM の中身を保存します (4.8 節)。次に起動すると、プログラムも変数もそのまま残っています。

保存したものを使わずにまったく新たな状態で始めたいときは --fresh をつけます。--fresh をつけたときは、終わるときにも保存しません。

```
$ uv run pc1251 --fresh
```

2.7 うまく動かないとき

cpu-1251.rom が見つかりませんと出る

ROM が置かれていません。2.4 節を見て、rom フォルダに置いてください。

uv というコマンドがないと言われる

uv を入れたあと、端末を開き直していません。閉じて開き直してください。

Windows で「スクリプトの実行が無効」と出る

uv を入れるコマンドを、上にしたとおり `-ExecutionPolicy Bypass` をつけて打ってください。

キーを打っても字が入らない

日本語入力になっていると、打ったキーがエミュレータに伝わりません。英数入力に切り替えてください。

Mac で F1～F12 を押しても反応しない

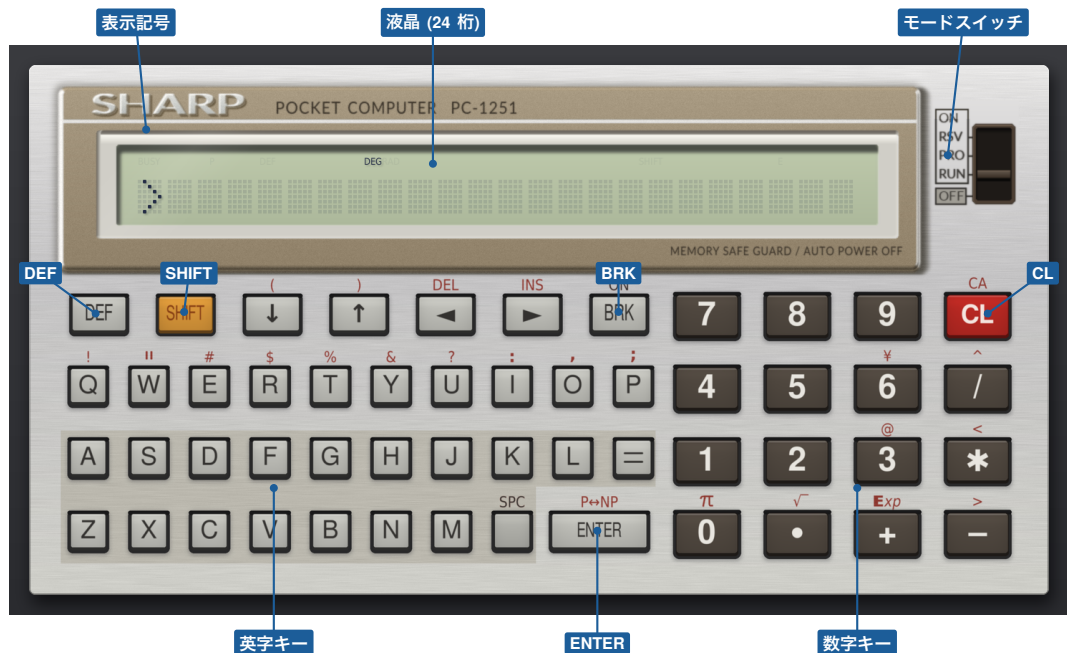
ノート型の Mac では、F1～F12 は明るさや音量のキーになっていることがあります。`fn` を押しながら押すか、3.4 節の `command` / `Ctrl` + `O` などの組み合わせや右クリックのメニューを使ってください。

音がいない

音を止めたいときは `--mute` をつけて起動します。

3 画面と操作

3.1 各部の名前



液晶

5×7 ドットの文字が 24 桁並びます。点いたドットは濃い青紫で、消えたドットもうっすら見えます。反射型の液晶なので、文字の影が奥に少しずれて落ちて見えます。

表示記号

液晶の上の小さな字です。DEG(角度の単位)、SHIFT(SHIFT を押した)、DEF、BUSY(計算中・実行中)、E(エラー) などが出ます。いまのモードは液晶には出ないので、スイッチの位置で見ます。

モードスイッチ

右上の縦のスイッチです。上から RSV、PRO、RUN、OFF。いちばん上の ON はスイッチの位置ではなく、**BRK** キーが ON(電源を入れる) を兼ねるという表示です。

3.2 マウスで操作する

- 画面のキーをクリックすると、本体の同じキーを押したことになります。ボタンを押しているあいだはキーも押しっぱなしです。
- モードスイッチは、ラベル (RSV～OFF) かつまみのあたりをクリックすると、クリックした位置に動きます。つまみを上下にドラッグしても動かさず。

3.3 パソコンのキーボードで操作する

英字・数字・**Enter**・**Space**・矢印キーは、PC-1251 の同じキーとして働きます。押しているあいだは押しっぱなしになります。記号は、パソコンで打った字を PC-1251 での打ち方 (**SHIFT** のあとにキー) に直して打ち込みます。たとえば **:** を打つと、**SHIFT I** と押したことになります。

PC-1251 の ROM は、1 つのキーを受け取るのに 0.1 秒ほどかかります (キーを離してから次のキーまでに、何も押されていない時間要る)。エミュレータは、パソコンで打ったキーを順番に並べて 1 つずつ届けるので、速く打っても取りこぼしません。毎秒 10 打鍵より速く打った分は、少し遅れて表示されます。連打しすぎて 0.6 秒以上待たせたキーは捨てるので、あとから遅れて流れてくることはありません。

macOS	Windows	PC-1251	説明
英数字・ Enter ・矢印 Space	同じ Space	同じキー SPC	押しているあいだは押しっぱなし
記号 Tab Option Delete (fn + delete) delete Esc	記号 Tab Alt Delete Backspace Esc	SHIFT + キー SHIFT DEF CL ◀ と DEL BRK	打った字に合わせて直す (付録付録 B) 表示を消す 1 字戻って消す 実行を止める。電源が切れているときは ON

3.4 エミュレータの操作

モードスイッチの切り替えや一覧の表示など、PC-1251 のキーにないエミュレータの操作は、**command** (Windows と Linux では **Ctrl**) との組み合わせで行います。

macOS	Windows	すること
command + ↑ ↓	Ctrl + ↑ ↓	モードスイッチを 1 段上・下へ動かす
command + O	Ctrl + O	プログラムの一覧 (5 節)
command + R	Ctrl + R	裏の RESET ボタンを押す
command + T	Ctrl + T	速くする (1/2 倍、1 倍、2 倍、4 倍、8 倍)
shift + command + T	Shift + Ctrl + T	遅くする
command + E	Ctrl + E	BASIC のプログラムをファイルに書き出す (5.9 節)
command + S	Ctrl + S	画面を画像で保存する
command + /	Ctrl + /	キーの説明を出す・消す
command + V	Ctrl + V	クリップボードの文字を打ち込む
command + K	Ctrl + K	矢印キーを数字キーの 8・2・4・6 にする・戻す

同じことは、ウィンドウの上で右クリックして出るメニューからもできます (Mac では **control** を押しながらクリックしても出ます)。メニューの項目はクリックするか、**↑** **↓** で選んで **Enter** で実行し

ます。いちばん上の「プログラムの一覧」を選ぶと、`command`/`Ctrl`+`O`と同じプログラムの一覧(5節)が開きます。



テンキーのないパソコンでゲームをするとき

PC-1251 のゲームには、数字キーの **8 2 4 6** (**5** を真ん中にした上下左右) で動かすものがあります。テンキーのないパソコンでは、`command`/`Ctrl`+`K` (または `F7`) を押すと、`↑` `↓` `←` `→` がそれぞれ **8 2 4 6** を押すようになります。もう一度押すと元に戻ります。プログラムを打ち込むときは、`↑` `↓` が PC-1251 の `↑` `↓` として使えなくなるので戻しておいてください。起動のときに `--numpad` を付けると、矢印キーが数字キーになった状態で始まります。

速さを変えたときの音

1 倍以外の速さでは、音の高さも速さの倍率だけ変わります。2 倍なら BEEP は 1 オクターブ高く聞こえます。8 倍では音が高くなりすぎて、本来と違う高さに聞こえることがあります。

ファンクションキー

`F1` ~ `F4` (スイッチの RUN、PRO、RSV、OFF)、`F5` (RESET)、`F6` (一覧)、`F7` (矢印キーを数字キーに)、`F9` (速くする。`Shift`+`F9` で遅くする)、`F10` (説明)、`F12` (画面の保存) も使えます。ただ、Mac では `fn` を押しながらでないと明るさや音量のキーになり、Windows でもほかのアプリに取られることがあるので、ふだんは上の組み合わせで右クリックのメニューを使ってください。

`command`/`Ctrl`+`S` で保存した画像は、ホームの `.pc1251` フォルダに `shot001.png`、`shot002.png`……の名前で置かれます (Windows では `C:\Users\名前\.pc1251`)。

3.5 モードスイッチ

モード すること

RUN	計算とプログラムの実行。
PRO	プログラムを打ち込む・直す・LIST で見る。
RSV	よく使う文字列をキーに登録する (4.6 節)。
OFF	電源を切る。プログラムと変数は残る。

3.6 SHIFT と DEF

SHIFT は、押して離してから次のキーを押します (同時に押す必要はありません)。押すと表示記号 **SHIFT** が点き、次のキーがキーの上に赤く刷ってある字になります。たとえば **SHIFT W** で"、**SHIFT 0** で π です。

DEF は、プログラムの中の"A"のような印の行から実行を始めるキーです (4.5 節)。

4 使ってみる

4.1 電卓として使う

RUN モードで式 (たとえば 12×3) を打って **ENTER** を押すと、答えが出ます。

12*3



CL (パソコンでは **Delete**) で表示が消えます。

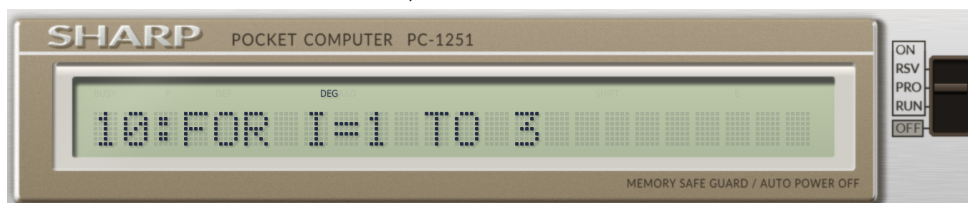
4.2 BASIC のプログラムを打ち込んで動かす

1. スイッチの PRO をクリックするか、**command**/**Ctrl**+**↑** で PRO モードにします。
2. 前のプログラムを消したければ **NEW** と打って **ENTER** を押します。
3. 行番号をつけて、1 行ずつ打ち込みます。行の終わりで **ENTER** を押します。

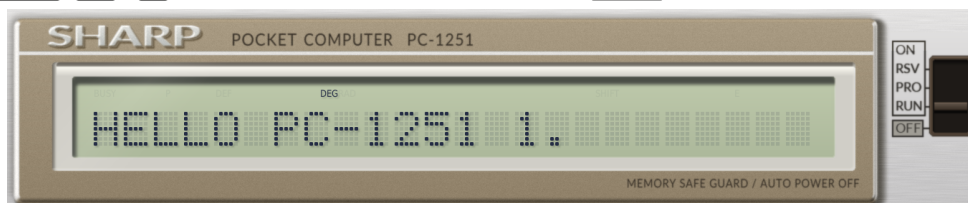
```
10 FOR I=1 TO 3
20 PRINT "HELLO PC-1251 ";I
30 NEXT I
```

```
40 END
```

4. **↓**を押すと先頭の行が出て、押すたびに次の行へ進みます。**↑**で前の行に戻ります (何も出ていないときに押すと、最後の行が出ます)。LIST 30 のように打つと、30 行から見られます。



5. `command/`**Ctrl**+**↓**で RUN モードに戻り、RUN と打って **ENTER** を押します。



PC-1251 の PRINT は、表示したところで止まって待ちます。**ENTER**を押すと先へ進みます (止めずに進める方法は次の節)。実行を途中で止めるには **BRK** (パソコンでは **Esc**) を押します。

長いプログラムを手で打つのは大変なので、ふつうはテキストファイルに書いておき、5 節の方法で読み込むのが楽です。

ENTER のあとは少し待つ

実機と同じく、**ENTER**を押したあとは ROM が行を中間コードに直すのに時間がかかります。直し終わる前に次のキーを押すと、取りこぼすことがあります。ファイルから読み込むときは、エミュレータが行ごとに直し終わるのを待ってから次の行を打ち込みます。

IF のあとの代入には LET がいる

IF の条件のあとに代入を書くときは、LET を省けません。THEN をつけても同じです。

書き方	結果
IF S>H LET H=S	代入される
IF S>H THEN LET H=S	代入される
IF S>H THEN H=S	ERROR 4
IF S>H H=S	エラーは出ないが、代入されない

最後の書き方は止まらずに進んでしまうので、間違いに気づきにくいところです。代入でなければ、IF A=1 GOTO 50 や IF D=1 POKE &C5DF,1 のように、条件のすぐあとに命令を書けます。

4.3 打ち込みを楽にする、行を直す・消す

■命令は略して打てる 命令の頭の何文字かのあとに.(ピリオド)を打つと、打った文字で始まる命令になります。たとえば P."AB"と打てば PRINT "AB"と同じです。PRO モードでも RUN モード

でも使え、LIST すると元の綴りで表示されます。

略記	命令	略記	命令	略記	命令
P.	PRINT	G.	GOTO	GOS.	GOSUB
I.	INPUT	F.	FOR	N.	NEXT
RE.	RETURN	REA.	READ	RES.	RESTORE
R.	RUN	L.	LIST	T.	THEN
B.	BEEP	W.	WAIT	CA.	CALL
D.	DIM	E.	END	S.	STOP

同じ文字で始まる命令がいくつかあるときは、ROM の命令の表で先に載っているものになります。C. は CALL ではなく CONT に、RE. は READ ではなく RETURN になるので、ほしい命令にならないときは文字を 1 つ増やします。.bas のファイルに略記で書いても同じように読み込め、5.9 節の方法で書き出すと元の綴りに戻ります。

■行を消す PRO モードで行番号だけを打って **ENTER** を押すと、打った番号の行が消えます。たとえば 20 **ENTER** で 20 行が消えます。プログラム全体を消すときは NEW です。同じ行番号で打ち直すと、前の行と入れ替わります。

■行を直す PRO モードで LIST 30 のように行を出し、**▶**を押すと、行番号のすぐあとの文字にカーソルが出ます。

- **◀▶** でカーソルを動かし、文字を打つとカーソルの位置の文字を上書きします。
- **SHIFT ▶** (INS) でカーソルの位置に 1 字ぶんの空きを作り、空いたところへ文字を打ち込みます。
- **SHIFT ◀** (DEL) でカーソルの位置の文字を消します。
- 直し終わったら **ENTER** を押すと、直した行に入れ替わります。
- **↓↑** で次の行・前の行を出せます。

■パソコンのキーボードで打つとき **Backspace** (macOS では **delete**) を押すと、エミュレータが **◀** と **SHIFT ◀** (DEL) を続けて押し、カーソルの 1 つ前の文字を消します。打ち間違えたらそのまま **Backspace** で戻れます。矢印キーは **◀▶↑↓**、**Delete** (macOS では **fn + delete**) は **CL** (表示を消す) です。

4.4 PRINT で止めずに見せる (WAIT)

PRINT のたびに **ENTER** を押すのでは、表示を動かすことができません。WAIT を使うと、PRINT は決まった時間だけ表示してから、止まらずに次へ進みます。

書き方	PRINT の動き
WAIT 64	表示してからおよそ 1 秒待ち、次へ進む。数は 10 進法で、大きいほど長く見せる
WAIT 0	待たずにすぐ次へ進む。表示は目に見えないほど短い
WAIT	数を書かないと、 ENTER を待つもとの動きに戻る

一度 WAIT を実行すると、以降の PRINT はすべて同じ待ち時間で動きます。次の例は、POCKET COMPUTER の文字を右から左へ 1 字ずつ流し、左端まで来たところで止まります (見本の anime.bas)。

```
10 DIM T$(0)*40
20 T$(0)="                                POCKET COMPUTER"
30 WAIT 2
40 FOR I=1 TO 24
50 PRINT MID$(T$(0),I,24)
60 NEXT I
70 WAIT:PRINT "POCKET COMPUTER"
80 END
```



ふつうの文字列変数 (A\$ など) には 7 字までしか入らないので、10 行の DIM T\$(0)*40 で、40 字まで入る文字列の配列 T\$ を作っています。かっこの中の 0 は添字の最大値で、使えるのは T\$(0) の 1 つだけです (T\$(1) を使うと ERROR 3 になります。DIM T\$(1)*40 なら T\$(0) と T\$(1) の 2 つが使えます)。*40 は 1 つに入る字数で、書かなければ 16 字です。20 行では、T\$(0) に空白を 24 字入れ、続けて流したい文字を入れておきます。50 行の MID\$ で I 字目から 24 字を切り出して表示し、I を 1 つずつ増やすと、文字が 1 字ずつ左へ動きます。30 行の WAIT 2 で、1 コマが 0.2 秒ほどになります (待ち時間のほかに、文字列の計算と表示の時間がかかる)。PC-1245 では左の 16 桁だけが見えるので、文字は見えない 17 桁目より右から入ってきます。

PC-1251 の BASIC には、液晶の点を 1 つずつ描く命令がありません。動きのある表示は、文字を PRINT し直して作ります。点で絵を描きたいときは、液晶の RAM に直接書くマシン語を使います (5.6 節の見本)。液晶に 1 列ずつ点を打てる PC-インタープリタ (5.7 節) の DEG も使えます。PC-インタープリタの PRINT は止まらないので、WAIT は時間待ちの命令として使います。

4.5 DEF キーで実行する

行の頭に "A" のような印をつけておくと、RUN モードで **DEF A** と押すだけで、印をつけた行から実行できます。

```
10 "A" PRINT 5
20 END
```

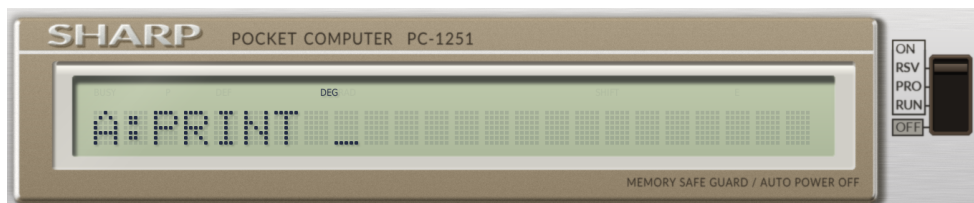
DEF A で 5 が出ます。印に使えるのは、RSV モード (4.6 節) と同じ灰色の帯の上の 18 個のキー

(**A**～**L**、**=**、**Z**～**M**、**SPC**) です。

4.6 RSV モードで文字列を登録する

よく打つ BASIC の命令や式を、キーに登録しておけます。

1. スイッチの RSV をクリックして RSV モードにします。
2. **SHIFT** **A** と押すと、液晶に **A:** と出ます。
3. 登録したい文字列 (たとえば **PRINT** や **12*3**) を打って、**ENTER** を押します。



4. RUN モードか PRO モードで **SHIFT** **A** と押すと、登録した文字列が打ち込まれます。

登録に使えるのは、キーボードの下 2 段の灰色の帯の上にある、**A**～**L**と**=**、**Z**～**M**と**SPC** の 18 個です。帯の外のキー (**Q** など) には登録できません。

4.7 エラーが出たら

打ち込んだ式やプログラムにまちがいがあると、**ERROR** と番号が出て、表示記号 **E** が点きます。

CL で消えます。



RUN モードで **PRINT 3** と打つと **ERROR 1** になる

RUN モードで **PRINT** を式の前につけて直接打つと、**ERROR 1** になります。エミュレータのまちがいではなく、ROM の動きです (ROM の 56BA 番地で状態を見て止めています)。電卓として使うときは **12*3** のように式だけを打ちます。プログラムの中の **PRINT** は、ふつうに動きます。

4.8 電源とメモリ

電源を切る

スイッチの **OFF** をクリックします。スイッチが **OFF** になったことを ROM が読んで、メモリが生きている印を書いてから自分で電源を切ります。

電源を入れる

スイッチを **RUN** か **PRO** に戻します。

オートパワーオフ

何もしないで 11 分ほどたつと、電源が切れます (実機と同じ)。**BRK** (ON) を押せば入ります。プログラムは残っています。

プログラムが残るしくみ

PC-1251 は、電源を入れたときに CPU 中のメモリに書いた印を確かめ、印がなければメモリがこわれたとみなして中身を消します。エミュレータは終わるときにも ROM に電源を切らせ、RAM(4KB) と CPU 中のメモリ (96 バイト) の両方を、ホームの `.pc1251/ram.bin` に保存します。

RESET ボタン

command / **Ctrl** + **R** で本体の裏の RESET ボタンを押せます。プログラムが暴走して **BRK** でも止まらないときに使います。

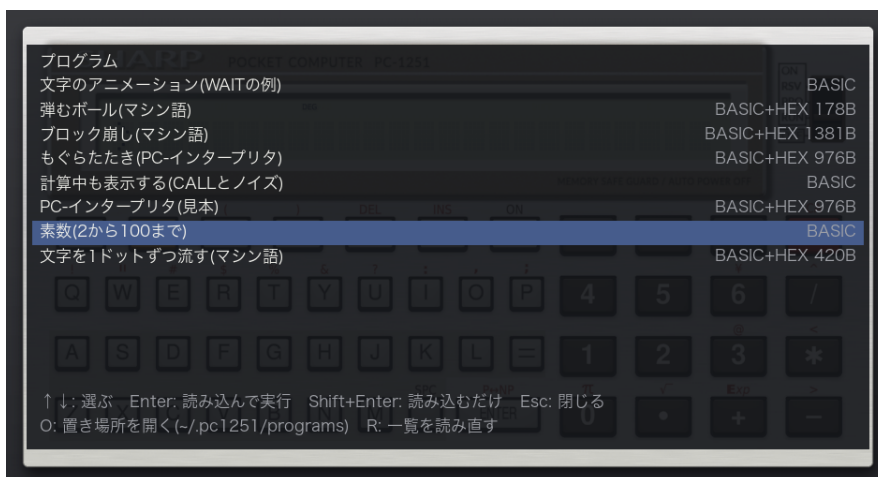
5 プログラムのファイルを使う

5.1 置き場所に置いて、一覧から読み込む

BASIC のプログラムやマシン語のダンプを、決まったフォルダ (置き場所) にファイルとして置いておくと、**command** / **Ctrl** + **O** で開く一覧に出てきます。

macOS	/Users/名前/.pc1251/programs
Windows	C:\Users\名前\.pc1251\programs

置き場所のフォルダは、最初に一覧を開いたときにできます。一覧で **O** を押すと、Finder やエクスプローラーで置き場所のフォルダが開くので、ファイルを入れてください。名前が「.」で始まるフォルダは、ふだんは隠れていて見えにくいので、**O** で開くのが確実です。



キー	すること
 	選ぶ (マウスでクリックしても選べる)
ホイール、  	一覧を動かす (入りきらないときは、右にバーと「▼」が出る)
	読み込んで実行する (選んであるものをもう一度クリックしても同じ)
 + 	読み込むだけ (打ち込んだプログラムを直したいときなど)
	置き場所のフォルダを開く
	一覧を読み直す (ファイルを足したあと)
	一覧を閉じる

置き場所のほかに、エミュレータのフォルダの `programs` にある見本 (素数、文字のアニメーション、計算中の表示、文字を 1 ドットずつ流す、弾むボール、ブロック崩し、PC-インタープリタ、もぐらたたき) も一覧に並びます。同じ名前のファイルが両方にあるときは、自分の置き場所のほうが使われます。

■機種ごとのフォルダ 置き場所の中に `pc1251` と `pc1245` のフォルダを作ると、`pc1251` に置いたものは PC-1251 用、`pc1245` に置いたものは PC-1245 用になります。置き場所の直下に置いたものは、どちらの機種でも使えます。見本も、マシン語を使うもののうち、ブロック崩し、文字を 1 ドットずつ流す、弾むボールは機種ごとのフォルダに入っています。

```

programs/
  primes.bas      どちらの機種でも
  anime.bas
  pc1251/
    break.bas    PC-1251 用
    break.hex
  pc1245/
    break.bas    PC-1245 用
    break.hex

```

一覧には、いま動かしている機種のフォルダと直下のものが並びます。ほかの機種のフォルダのものは「(PC-1251 用)」のように添えて灰色で出し、選んでも読み込みません。ただし、同じ `title` のものがいまの機種のフォルダにあれば、いまの機種のものだけを出します。機種ごとに作り分けたプログラムは、同じ名前でも両方のフォルダに置いておけば、どちらの機種でも一覧の同じ名前から読み込めます。

■ドラッグ&ドロップ Finder やエクスプローラーからファイルをエミュレータのウィンドウにドロップしても、同じように読み込めます。

5.2 ファイルの形式

ファイル	読み込み方
名前.bas	BASIC のテキスト。PRO モードにして NEW してから 1 行ずつ打ち込み、もとのモードに戻る。
名前.hex	16 進ダンプ。番地 データの行を読んで、RAM にそのまま書き込む。
名前.bas と名前.hex	同じ名前の 2 つは 1 本のプログラムとして扱い、ダンプを書いてから BASIC を打ち込む。マシン語と、マシン語を呼ぶ BASIC を組にしておける。

■BASIC のファイル ふつうのテキストファイルです。文字コードは UTF-8(ASCII の範囲なら何でも同じ)で、改行は macOS・Windows どちらの形でもかまいません。小文字で書いても大文字に直して打ち込みます。PC-1251 のキーにない字は捨てます。指数の E(**SHIFT** + で打つ Exp の字。英字の E とは別の字です) は [E] か **€** と書き、1[E]-8 のようにします (PocketTools と同じ書き方)。 π (**SHIFT** 0) は π と書きます。空の行と、#で始まる行は打ち込みません。1 行に打てるのは行番号を含めて 79 字までなので、それより長い行は実機と同じように、途中まで打って **ENTER** してから行末に続きを足します。それでも直す欄からあふれる行は、命令の綴りを先に打ち、残りの字をあとから **SHIFT** **▶** (INS) で途中に挿入して入れます。

```
# title: 素数 (2 から 100 まで)
# PRINT で止まるので、ENTER で次を出す
10 PRINT 2
20 FOR N=3 TO 100 STEP 2
30 D=3
40 IF D*D>N GOTO 70
50 IF N=INT (N/D)*D GOTO 80
60 D=D+2:GOTO 40
70 PRINT N
80 NEXT N
90 END
```

■16 進ダンプのファイル 1 行に番地とデータを 16 進法で書きます。番地は 4 桁です。データは空白で区切っても続けて書いてもかまいません。雑誌のダンプリストのように行の終わりに: とチェックサムがついていれば、: から後ろは読み飛ばします。#か; で始まる行はコメントです。行の途中で; から後ろはコメントなので、ニーモニックを書き添えておけます。

```
# title: マシン語の見本
C300 02 41 37
C303 0102030405060708 ; 表の中身
```

アセンブラ YASM61860 が出す 16 進ダンプ (c300 : 02 41 37 ...のような行) も、そのま

ま.hexとして読めます。YASM61860は行の番地を8の倍数にそろえ、行の途中から始まる所は手前を空白で埋めて出すので、その空白の分だけ番地を進めて読みます。

■マシン語のバイナリ(.bin) PocketToolsのbin2wavで使う.binは、マシン語のバイトが並んでいるだけで、番地を持ちません。.basに# bin: ファイル名 番地の行を書いて、置く番地を与えて読み込みます。番地は&C300、0xC300、C300のどれで書いてもかまいません。

```
# title: マシン語の見本 (バイナリ)
# bin: sample.bin &C300
10 CALL &C300
```

YASM61860の-rが出す.binは、番地0からの中身です(ORGより前は0で埋めてあります)。そのまま# bin: で読むとRAMの作業場所まで0で埋まるので、YASM61860で作ったものは、ダンプ(-d)を.hexとして使ってください。

■指示の行 #で始まる行には、次の指示を書けます。.basと.hexのどちらに書いてもかまいません。

書き方	意味
# title: 名前	一覧に出す名前。書かなければファイル名。
# run: 文字列	一覧から選んだときや--runで実行するときに、RUNモードで打つ文字列。書かなければRUN。
# hex: ファイル名	先に書き込む16進ダンプ。同じフォルダ、置き場所の同じ機種種のフォルダ、置き場所の直下の順に探す。1つのマシン語を、いくつものBASICのプログラムから使える。
# bin: ファイル名 番地	先に書き込むマシン語のバイナリ(.bin)と、置く番地。探す順は# hex: と同じ。
# after: ファイル名	実行の文字列を打ったあとで書き込む16進ダンプ。BASICでDIMなどをして場所を空けてからマシン語を読み込む記事のためのもの。# after: ファイル名.bin 番地と書けば.binも使える。
# model: 機種	プログラムが動く機種(1251、1245。いくつもあるときは、で区切る)。ほかの機種で動かしているときは、一覧に「(PC-1251用)」のように出て、読み込まない。同じtitleでいまの機種用のものがあれば、いまの機種用のものだけが一覧に出る。書かなければ置いたフォルダで決まる(直下ならどの機種でも)。

hex: や# after: で使われるだけの.hex(同じ名前の.basがないもの)は、一覧には出ません。.binも一覧には出ません。ダンプの大きさは、使う側のプログラムの行に出ます。

読み込みの速さ

BASICは、実機でキーを押すのと同じ打ち込み方で入れます(行を中間コードに直すのはROMの仕事です)。読み込むあいだは実機の時計に合わせず全速で動かすので、数十行のプログラムでも1~2秒で終わります。画面の下に進み具合が出ます。

NEW で前のプログラムは消える

.bas を読み込む前に NEW するので、入っていたプログラムは消えます。残しておきたいプログラムは、先に 5.9 節の方法でファイルに書き出しておいてください。

5.3 端末から読み込む

起動するときにプログラムの名前かファイルの場所を書くと、起動してすぐに読み込みます。名前だけなら置き場所から探します。--run をつけると、読み込んだあとと実行します。

```
$ uv run pc1251 primes --run
$ uv run pc1251 ~/basic/game.bas
```

Windows なら、ファイルの場所は C:\Users\名前\basic\game.bas のように書きます。

5.4 マシン語を動かす

マシン語は 16 進ダンプのファイルにして RAM に書き込み、BASIC の CALL で呼びます。PC-1251 の RAM は B800~C7FF 番地の 4KB で、BASIC のプログラムは下の番地から、変数は上の番地から使います。マシン語は、BASIC のプログラムや変数とぶつからない番地に置いてください。

5.5 CALL で表示をつけたまま計算する

BASIC はプログラムを実行しているあいだ、液晶の表示を切っています (C ポートのビット 0 を 0 にする)。マシン語でビット 0 を 1 にしてから計算を続けると、実行中も表示が消えません。見本の noise.bas は、表示をつけるマシン語 5 バイトを POKE で書いて CALL で呼び、計算しながら液晶に棒を描きます。

```
10 POKE &C200,2,1,255,177,55
20 CALL &C200
30 FOR I=0 TO 47
40 B$=STR$ (I*I)
50 POKE &F87B-I,127
60 NEXT I
70 WAIT 128:PRINT "DONE"
80 END
```

10 行で書くマシン語は次の 3 命令です。1FB1 番地は CPU の内部 ROM にあるルーチンで、A の値を C ポートに出します。

C200	02 01	LIA 01	A=&01(表示をオン)
C202	FF B1	CAL 1FB1	C ポートに A を出す

C204	37	RTN	BASIC に戻る
------	----	-----	-----------

50 行は、液晶 RAM に直接 1 列ずつ点を書きます。液晶の右半分は F87B 番地が左端 (60 列目) で、番地が小さくなるほど右へ進むので、&F87B-I で棒が右へ伸びます。



表示をつけたままにすると、左の数桁に不規則な点の模様が出ます。ROM が液晶 RAM の一部を計算の作業場所に使っているためです。40 行の STR\$ は数を文字にする途中で F800~F822 番地 (左から 7 桁ほど) を書き換え、ほかの計算でも右端の F840~F847 番地などを使います。ふだんは実行中に表示を切っているため、作業場所の書き換えは目に見えません。

CALL するマシン語の置き場所

C200 番地は PC-インタープリタ (5.7 節) の置き場所の中にあり、C000 番地から置くマシン語の見本とも重なります。続けて使うときは、あとから読み込んだ方が残ります。

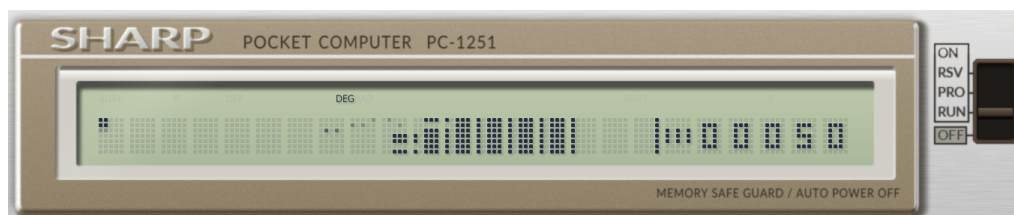
5.6 マシン語で液晶に描く

■文字を 1 ドットずつ流す 見本の scroll.hex と scroll.bas は、POCKET COMPUTER と機種名を、液晶の全幅で右から左へ 1 列ずつ流すマシン語です。4.4 節のように PRINT で流すと 1 文字 (5 列) ずつ跳びますが、液晶 RAM に直接書けば 1 列ずつ動かします。文字の形はマシン語の後ろの表に持っていて、1 コマごとに表を読み始める場所を 1 列ずらし、液晶の幅ぶんを液晶 RAM へ写します。液晶は点が消えるのが遅く、細い線は流すときにじむので、線の太さ 2 列の太い文字にしてあります。液晶の 5 列ごとのすき間で文字が切れて見えるのは、実機でも同じです。.hex には 1 行ごとに ; のあとへニーモニックを書いてあります。同じ名前の scroll.bas が CALL で呼ぶので、一覧から選ぶと両方が読み込まれます。Esc (BRK) を押すと BASIC に戻ります。PC-1251 用は C000 番地から、PC-1245 用は C100 番地から置きます。液晶の幅が違うので、2 つは別々のマシン語です。

■弾むボール 見本の ball.hex と ball.bas は、ボールが地面でひしゃげ、放物線を描いて弾みながら、液晶の全幅を右へ進み、右端まで行くと左端から出てくるマシン語です。1 回の弾みを 9 枚の絵にして表に持ち、1 枚ずつ液晶 RAM へ写します。着地の直前と跳ね上がった直後は縦に伸び、地面では横にひしゃげます。着地の前後の 3 枚は同じ桁に置きます。縦に伸びた絵は、着地の直前を 1 列左に、跳ね上がった直後を 1 列右に寄せ、ボールが斜めに入って斜めに出るようにしてあります。液晶は 5 列ごとに桁のすき間があり、ボールがすき間をまたぐと割れて見えるので、どの絵も 1 桁の枠に収めます。液晶の右半分は番地が逆向きに並ぶので、絵を描く桁が右半分にあるときは番地を減らしながら書きます。PC-1251 用は C000 番地から、PC-1245 用は C100 番地から置き、違うのは液晶の幅だけです。

■ブロック崩し 見本の pc1251/break.hex と break.bas は、横向きのブロック崩しです。左端のパドルを↑↓で動かし、SpaceかEnterで玉を打ち出して、右のブロックの壁を削ります。壁と画面の右端のあいだは空いているので、空いたところへ玉を回り込ませると壁を裏から削れます。壁はゆっくり左へ寄ってきて、先端が左から 12 列目まで来ると押し切れ、玉を 1 個失います。玉は 3 個で、全部なくなると BASIC に戻って得点を出します。右端の 3 つの点が残りの玉、残りの玉の右の 5 桁が得点です。

始めに 1:EASY 2:NORMAL ?と聞いてくるので、1か2を押してENTERを押します。EASY はパドルが縦 3 段で、玉は 1 コマに 1.5 列、壁は 60 コマで 1 列と遅めです。NORMAL はパドルが縦 2 段で、玉は 1 コマに 2 列、壁は 40 コマで 1 列です。ENTERだけを押しと前と同じ難易度で始めます。最高点は難易度ごとに覚えています。



パドルに当たった場所で玉の角度が変わり、パドルを動かしながら当てるとさらに角度がつきます。縦が 7 ドットしかないので、同じ角度のまま打ち返していると、同じ段ばかりを往復して最後の数個が残りがちです。角度を変えて狙ってください。面をクリアすると 500 点が入り、次の面から玉が少し速くなります。

面ごとに 1 つ、点減しているブロックがあります。点減しているブロックを壊すと、壊した場所からもう 1 個の玉が出て、2 個で遊べます。玉は 2 個までです。新しい玉は壊した玉と同じ向きに進み、壊した玉のほうに跳ね返るので、2 個はブロックの場所から別々の向きへ進みます。片方を落としても残りの玉は減らず、2 個とも落とすと 1 個失います。

マシン語は C000~C56E 番地の約 1.4KB(表は C501 番地から)で、液晶に描く前の絵を BF88 番地からの 120 バイトに組み立ててから、液晶の RAM へまとめて写します。2 個目の玉の位置と向きは BF84 番地からの 4 バイトに置きます。BASIC のプログラムは BF83 番地までに収まっている必要があります (break.bas は B920 番地で終わります)。ROM のルーチンは呼ばず、キーは IA・IB のポートを直接読み、音は C ポートのビット 4~6 を切り替えて出します。内部 RAM の 10h~2Fh を変数に使うので、入るときに BASIC の値を C5E0 番地からの 32 バイトへ退避し、出るときに入れ替えて戻します。終わったあとも変数は退避先に残るので、得点の 5 桁は C5F6~C5FA 番地から PEEK で読めます。break.bas は退避先の得点を読んで、得点と最高点を出します。難易度は、CALL の前に C5DF 番地へ POKE しておきます (1 なら EASY)。PC-1245 用 (6 節) も、得点の退避先と難易度の番地は同じです。

5.7 PC-インタープリタともぐらたたき

見本には、PiO 1986 年 8 月号の「PC-インタープリタ」(Fan_PC-1251 氏作)が入っています。BASIC のエディタで書いた行を独自の文法で読んで実行するマシン語のプログラムで、数はすべて 16 進法 2 桁の 1 バイト、命令は PRINT、IF～NOT、GOTO、FOR～NEXT、BEEP、WAIT、液晶に 1 列ずつ点を打つ DEG などです。

pcint.hex と pcint.bas

記事のリスト 1(C200～C5CF の 976 バイト) と、PC-インタープリタを試す短いプログラムの組です。一覧から選ぶと、CALL &C300 で動き出します。

mogura.bas

記事のリスト 2 のもぐらたたきです。# hex: pcint.hex の行で PC-インタープリタを先に書き込み、RUN 900 で始めます。



もぐらたたきを始めるには、`command`/Ctrl+O(または右クリックのメニューの「プログラムの一覧」)で開く一覧から「もぐらたたき (PC-インタープリタ)」を選びます。PC-インタープリタのダンプとリスト 2 が読み込まれ、RUN モードで RUN 900 が打たれてゲームが始まります。一度読み込んだあとは、RUN モードで RUN 900 と打つか、DEF = を押せば何度でも遊べます (900 行の頭に "=" の印がある)。

900 行は CALL &C300 で PC-インタープリタを呼び、ゲームが終わって BASIC に戻ってから PRINT で得点を出します。CALL &C300 とだけ打っても遊べますが、得点は出ません。

遊び方は次のとおりです。液晶に 1～6 の穴が並び、どこかの穴からもぐらが 3 段階で顔を出します。出た穴の数字キーを押しっぱなしにしておくと、当たりの音が鳴って得点が 1 つ増え、次のもぐらが出ます。当てるたびに少しずつ速くなります。3 段階まで出きても押せないと GAME OVER になり、ENTER を押すと得点が出ます。

記事のプログラムの扱い

pcint.hex と mogura.bas は記事の作者 (Fan_PC-1251 氏) の著作物で、作者の了解を得て収めています。PC-インタープリタを 1 命令ずつ解説した『PC-インタープリタを読む』(おりぐち) も doc/pcinterp.pdf にあります。

5.8 ほかの読み込み方

`command`+V (Windows は Ctrl+V)

クリップボードの文字を、そのままキーとして打ち込みます。NEW もモードの切り替えもしないの

で、打ち込む前に PRO モードにしておきます。

--type ファイル

起動したあと、ファイルの中身をそのままキーとして打ち込みます。

--load ファイル

起動したあと、16 進ダンプを RAM に書き込みます。

5.9 プログラムをファイルに書き出す

エミュレータで打ち込んだり直したりした BASIC のプログラムは、RAM の中にしかありません。

`command` / `Ctrl` + `E` を押すか、右クリックのメニューで「プログラムをファイルに書き出す」を選ぶと、いまのプログラムをプログラムの置き場所 (`~/ .pc1251/programs`) に `.bas` のファイルとして書き出します。ファイル名は「名前-日時.bas」で、名前は最後に読み込んだプログラムのもの (なければ `program`) です。最後に読み込んだものが機種ごとのフォルダにあったときは、置き場所の同じ名前のフォルダ (`pc1245` など) に書き出します。書き出したファイルはすぐ一覧に出るので、そのまま読み込み直せます。

RAM のプログラムは中間コードになっているので、ROM の中にある命令の表を引いて文字に戻します。空白は ROM が打ち込みのときに捨てているので、命令の前後に読みやすいように補います。打ち込み直せば、元と同じ中間コードになります。

読み込んだプログラムに組になる `.hex` があったときは、書き出したファイルの頭に `# hex:` の行を入れて、元のダンプを指しておきます。 `# bin:` で読んだ `.bin` も、同じ行を入れておきます。POKE で書いたマシン語など RAM の中身そのものは書き出しません。

エミュレータを起動しないで、保存してある RAM から書き出すこともできます。

```
$ uv run pc1251 --export ~/Desktop
```

5.10 カセットの音 (wav) で実機とやり取りする

実機のカセットインターフェースで読み書きする音を、wav のファイルにして実機とやり取りできます。

■エミュレータから実機へ RUN モードで `CSAVE` か `CSAVE M` を打つと、ROM がカセットに出す音を、エミュレータがそのまま wav のファイルに書き出します。ファイルはプログラムの置き場所 (`~/ .pc1251/programs`) に「tape-日時.wav」の名前で置かれ、画面の下にファイルの名前が出ます。一覧で `O` を押すと、置き場所のフォルダを開けます。実機に読み込ませるときは、wav をパソコンで再生し、実機のカセットインターフェースの入力につないで、実機で `CLOAD` (マシン語は `CLOAD M`) を打ちます。

```
CSAVE "BREAK"  
CSAVE M "BREAK";&C000,&C56E
```

エミュレータは、4kHz と 2kHz の音を切り替えるカセットの信号が 2 秒以上続いたときだけ、音が止まって 1 秒たったところで wav を書き出します。BEEP の音は書き出しません。前置きの音が 8 秒あるので、書き出しまでには、どんなに短いプログラムでも 10 秒ほどかかります (`command` / `Ctrl` + `T` で速くすると短くなります)。

■実機からエミュレータへ 実機で CSAVE した音をパソコンで録音して wav にし、ウィンドウにドロップすると、テープをセットした状態になります (起動のときに `--tape` ファイルを付けても同じ)。テープをセットしたあと、RUN モードで CLOAD か CLOAD M を打つと、ROM がテープを読み始めたところで再生が始まり、実機と同じ時間をかけて読み込みます (前置きの音だけで 8 秒、ブロック崩しのマシン語は 1 分ほど。 `command` / `Ctrl` + `T` で速くもできます)。

実機で読ませてはいません

実機からエミュレータへの向きは、PC-1251 の見本プログラムのマイクロカセットを録音した wav (GitHub の [number42net/sharp-pc1251](#) で公開されている 20 本) で確かめました。どれも CLOAD でエラーなく読めます。PocketTools の bin2wav や、YASM61860 の `-w(-old` 付き) で作ったマシン語の wav も、両機種で CLOAD M で読めます。

エミュレータの CSAVE で書き出した wav は、PocketTools の wav2bin で元どおりに読めることまで確かめましたが、実機で読ませてはいません。

6 PC-1245 として使う

PC-1245 は、PC-1250/1251 と同じ大きさの筐体です。CPU は同じ SC61860 ですが、ROM は PC-1251 とは別のものです。起動のときに `--model 1245` を付けると、PC-1245 として動きます。

```
$ uv run pc1251 --model 1245
```



ROM は、実機から読み出したものを `cpu-1245.rom` と `bas-1245.rom` の名前で `rom` フォルダに置きます。読み出し方を書いたページも 2.4 節で紹介しています。

.bas や .hex のファイルを使う (5 節)、書き出す (5.9 節)、`command`/`Ctrl`+`O` などのショートカット (3.4 節)、`DEF` キーでの実行 (4.5 節) は、PC-1251 と同じように使えます。違うのは次のところです。

6.1 液晶

液晶は 16 桁です。PC-1251 と同じ液晶 RAM のうち、左の 12 桁 (F800~F83B) と、右側の初めの 4 桁 (F87B~F868) だけが見えます。17 桁目より右に書いたものは表示されないの、PC-1251 用のプログラムには表示が途中で切れるものがあります。

表示記号は BUSY、DEF、SHIFT、P、DE・G・RAD(DEG などの角度の単位)、E です。モード (PRO、RUN) は液晶に出ず、スイッチの位置で見ます。表示記号の位置は、実機の写真で見える DEG のほかは推定です。

液晶の右には、容量の表 (RAM 2.2K BYTE、1486 STEPS、26 MEMORIES) と、空きの調べ方が刷ってあります。MEM で残りのステップ数が、 $(MEM-6)/8$ で残りの変数の数がわかります。

6.2 モードスイッチ

スイッチは PRO、RUN、OFF の 3 段です。PRO と RUN の 2 段は「ON」の括りに入っていて、電源が入る位置です。RSV モードはないので、4.6 節の文字列の登録は使えません。`F3` は何もせず、`command`/`Ctrl`+`↑`・`command`/`Ctrl`+`↓` と右クリックのメニューも、PRO、RUN、OFF の 3 段のあいだだけを動きます。

6.3 キー

キーの位置は PC-1251 と同じですが、刷ってある文字が違います。

- **A**～**=**の段と**Z**～**SPC**の段は、**SHIFT**を押してから打つと、キーの上に刷ってある BASIC の命令が入ります。たとえば**SHIFT Z**で PRINT、**SHIFT G**で FOR、**SHIFT =**で RUN、**SHIFT SPC**で CLOAD です。
- 括弧は**SHIFT 1**が (、**SHIFT 2**が) です (PC-1251 は ↓ と ↑)。パソコンで (、) を打つと、エミュレータが**SHIFT 1**、**SHIFT 2**に直して打ちます。貼り付けやファイルの読み込みでも同じです。
- ほかの記号は付録付録 B の表と同じです。

6.4 メモリと見本のプログラム

RAM は 2KB(C000～C7FF) で、BASIC のプログラムは C000 番地から置かれます。プログラムに使えるのは 1486 ステップ (バイト) です。終わるときの RAM は、PC-1251 の ram.bin とは別の ~/.pc1251/ram-1245.bin に保存します。

見本のプログラムは同じ一覧から読み込めますが、次の点に気をつけてください。

- ブロック崩し (5.6 節) は、programs/pc1245 にある PC-1245 用の break.hex と break.bas が読み込まれます。遊ぶ場所は 75 列で、右端の桁に残りの玉が出ます。遊んでいるあいだは得点を出さず、終わってから出します。点減するブロックはなく、玉はいつも 1 個です。マシン語は C100～C55D 番地で、BASIC のプログラムは C000～C0FF に収まるようにしています。
- PC-インタープリタは C200～C5CF に入ります。PiO の解説のとおり、PC-1245 で残るフリーエリアは 510 バイトです。
- ほかの見本は動きますが、24 桁を使う表示は 17 桁目から右が見えません。

付録 A 起動のオプション

オプション	意味
プログラム (名前)	読み込むプログラム (ファイルの場所か、置き場所にある名前)
--run	読み込んだあと実行する
--scale 数	拡大率 (既定は 1.0。画面に収まらなければ縮める)
--fresh	保存した RAM を読まず、終わるときにも保存しない
--mute	音を出さない
--persist 数	液晶の残像の強さ (既定は 1.0、0 で残像なし)
--type ファイル	起動後にファイルの中身をキーとして打ち込む
--load ファイル	起動後に 16 進ダンプを RAM へ書き込む
--export フォルダ	保存した RAM の BASIC のプログラムを .bas で書き出して終わる
--model 機種	1251(既定) か 1245。PC-1245 として動かすときは 1245(6 節)
--numpad	矢印キーを数字キーの 8・2・4・6 として押す状態で始める (command / Ctrl + K で切り替え)
--tape ファイル	カセットの音 (wav) をセットして始める (5.10 節)
--help	オプションの一覧を出す
--version	版を出して終わる

環境変数	意味
PC1251_ROM	ROM を置いたフォルダ
PC1251_PROGRAMS	置き場所として先に探すフォルダ。いくつも書くときは、macOS では:、Windows では; で区切る

付録 B SHIFT で入る記号

パソコンで次の記号を打つと、右のキー操作に直して打ち込みます。

記号	キー	記号	キー	記号	キー
!	SHIFT+Q	&	SHIFT+Y	(	SHIFT+ ↓
"	SHIFT+W	?	SHIFT+U	)	SHIFT+ ↑
#	SHIFT+E	:	SHIFT+I	@	SHIFT+3
\$	SHIFT+R	,	SHIFT+O	<	SHIFT+*
%	SHIFT+T	;	SHIFT+P	>	SHIFT+-
^	SHIFT+/	√	SHIFT+.	π	SHIFT+0
€	SHIFT++				

PC-1245 では (と) が SHIFT+1 と SHIFT+2 になります (6 節)。€ は指数の E(Exp) です。貼り付けやファイルでは [E] と書いてもかまいません。

SHIFT + は指数の E(Exp)、SHIFT ◀ は 1 字消す (DEL)、SHIFT ▶ は 1 字あける (INS)、SHIFT ENTER は P ↔ NP(プリンタを使うかどうかの切り替え)、SHIFT CL は CA(すべて消す) です。

付録 C メモリマップと入出力

番地	中身
0000-1FFF	CPU の内部 ROM(8KB)
4000-7FFF	BASIC ROM(16KB)
B800-C7FF	RAM(4KB)。B000-B7FF は B800-BFFF と同じものが見える。PC-1245 は C000-C7FF(2KB)
F800-F87F	液晶の RAM。F800-F8FF と同じものが F900-FFFF の 256 バイトごとに見える。PC-1245 は E800-EFFF にも見える

- CPU は SC61860 で、命令サイクルは 192kHz(水晶 576kHz の 1/3) です。命令の意味は MAME の実装をもとにしましたが、CUP、CDN、TEST など、ROM や当時のプログラムの動きと合わない命令は変えています。CUP、CDN とタイマの印の扱いは、PockEmul と PokecomGO の実装とも見比べて確かめました。
- モードスイッチは、ポート IB のビット 3 を出して INB 命令で読みます。ビット 0 が RSV、ビット 1 が PRO、ビット 2 が OFF で、どれも立っていなければ RUN です。
- C ポートのビット 2 を立てると、CPU はキーが押されるか 512ms のタイマが変わるまで止まって待ちます。ROM はキーを待つあいだ、C ポートのビット 2 を立てて止まり、起きた回数を数えてオートパワーオフの時間を計ります。
- ブザーは C ポートのビット 4~6 で鳴らします。BASIC の BEEP は内蔵の 4kHz の発振を使い、マシン語ではビットを直接切り替えて好きな高さの音を出せます。

付録 D 実機と違うところ

- 実機と並べて動きを比べてはいません。命令の意味、キーの配置、ポートの使い方は、MAME と PockEmul のソース、digihoori 氏の PokecomGO(<https://github.com/digihoori/pokecom>) のソース、POPCOM 1984 年 9~11 月号のマシン語講座を参考にし、わからないところは ROM のコードを読んで決めました。
- カセットテープの入出力は、実機のテープを録音した wav を CLOAD で読めるところまで確かめました。エミュレータが書き出した wav を実機で読ませてはいません (5.10 節)。
- プリンタ (CE-125) は動きません。
- ROM の版の違いは確かめていません。
- 内部 ROM が BASIC の PEEK でも読めます。実機では DATA(35h) 命令でしか読めません。

付録 E ファイルの一覧

場所	中身
pc1251emu/	エミュレータ本体 (CPU、メモリとポート、画面、音、プログラムの読み込み)
programs/	見本のプログラム (機種ごとのものは pc1251/ と pc1245/)
rom/	自分で用意した ROM(2.4 節、PC-1245 は 6 節)
tests/	動作の試験 (uv run python tests/test_basic.py など)
doc/	この手引きと『PC-インタプリタを読む』(pcinterp.pdf)
~/.pc1251/	保存した RAM、画面の画像、プログラムの置き場所